

JAVA PRACTICAL RECORD

Exception Handling Programs

Experiment No.	03
Date	17/09/2026
Package	observation.exp03
Programs	10

This practical record contains ten Java programs demonstrating common runtime exceptions and their handling using try-catch blocks.

Experiment 03 — Program 1/10

ArrayIndexOutOfBoundsException

1. Objective / Aim

Access an array using an invalid index and handle ArrayIndexOutOfBoundsException.

2. System Requirements

Hardware: Standard computer/laptop. Software: Java Development Kit (JDK 8 or later), command prompt/terminal, and a Java-compatible editor or IDE.

3. Algorithm / Logic Description

- Start the program.
- Place the statement that may cause the exception inside a try block.
- Execute the statement that produces ArrayIndexOutOfBoundsException.
- Catch ArrayIndexOutOfBoundsException using the corresponding catch block.
- Display a suitable message confirming that the exception was handled.
- Stop the program.

4. UML Class Diagram

ArrayIndexOutOfBoundsException
main(String[] args)
try { exception-causing statement }
catch (ExceptionType e)
print handled message

5. Source Code

```
package observation.exp03;

public class ArrayIndexOutOfBoundsException {
    public static void main(String[] args) {

        try {
            int ar[] = {10, 20, 30};

            System.out.println(ar[3]);
        }
        catch (ArrayIndexOutOfBoundsException aiobe) {
            System.out.println("Cannot access an array using an invalid index.");
            System.out.println("ArrayIndexOutOfBoundsException is handled successfully.");
        }
    }
}
```

6. Compilation & Execution Commands

```
javac -d . ArrayIndexOutOfBoundsException.java
java observation.exp03.ArrayIndexOutOfBoundsException
```

7. Output

```
Cannot access an array using an invalid index.
ArrayIndexOutOfBoundsException is handled successfully.
```

8. Result

The program successfully demonstrates handling of `ArrayIndexOutOfBoundsException` using exception handling in Java.

Experiment 03 — Program 2/10

ArrayStoreException

1. Objective / Aim

Store an incompatible value in an array and handle ArrayStoreException.

2. System Requirements

Hardware: Standard computer/laptop. Software: Java Development Kit (JDK 8 or later), command prompt/terminal, and a Java-compatible editor or IDE.

3. Algorithm / Logic Description

- Start the program.
- Place the statement that may cause the exception inside a try block.
- Execute the statement that produces ArrayStoreException.
- Catch ArrayStoreException using the corresponding catch block.
- Display a suitable message confirming that the exception was handled.
- Stop the program.

4. UML Class Diagram

ArrayStoreException
main(String[] args)
try { exception-causing statement }
catch (ExceptionType e)
print handled message

5. Source Code

```
package observation.exp03;

public class ArrayStoreException {
    public static void main(String[] args) {

        try {
            Object arr[] = new String[3];

            arr[0] = 100;
        }
        catch (ArrayStoreException ase) {
            System.out.println("Invalid type of value stored in the array.");
            System.out.println("ArrayStoreException is handled successfully.");
        }
    }
}
```

6. Compilation & Execution Commands

```
javac -d . ArrayStoreException.java
java observation.exp03.ArrayStoreException
```

7. Output

```
Invalid type of value stored in the array.
ArrayStoreException is handled successfully.
```

8. Result

The program successfully demonstrates handling of `ArrayStoreException` using exception handling in Java.

Experiment 03 — Program 3/10

ClassCastException

1. Objective / Aim

Attempt an invalid type cast and handle ClassCastException.

2. System Requirements

Hardware: Standard computer/laptop. Software: Java Development Kit (JDK 8 or later), command prompt/terminal, and a Java-compatible editor or IDE.

3. Algorithm / Logic Description

- Start the program.
- Place the statement that may cause the exception inside a try block.
- Execute the statement that produces ClassCastException.
- Catch ClassCastException using the corresponding catch block.
- Display a suitable message confirming that the exception was handled.
- Stop the program.

4. UML Class Diagram

ClassCastException
main(String[] args)
try { exception-causing statement }
catch (ExceptionType e)
print handled message

5. Source Code

```
package observation.exp03;

public class ClassCastException {
    public static void main(String[] args) {

        try {
            Object obj = "Hello";

            Integer num = (Integer) obj;
        }
        catch (ClassCastException cce) {
            System.out.println("Cannot convert String to Integer.");
            System.out.println("ClassCastException is handled successfully.");
        }
    }
}
```

6. Compilation & Execution Commands

```
javac -d . ClassCastException.java
java observation.exp03.ClassCastException
```

7. Output

```
Cannot convert String to Integer.
ClassCastException is handled successfully.
```

8. Result

The program successfully demonstrates handling of ClassCastException using exception handling in Java.

Experiment 03 — Program 4/10

IllegalArgumentException

1. Objective / Aim

Pass an invalid argument to Integer.parseInt() and handle IllegalArgumentException.

2. System Requirements

Hardware: Standard computer/laptop. Software: Java Development Kit (JDK 8 or later), command prompt/terminal, and a Java-compatible editor or IDE.

3. Algorithm / Logic Description

- Start the program.
- Place the statement that may cause the exception inside a try block.
- Execute the statement that produces IllegalArgumentException.
- Catch IllegalArgumentException using the corresponding catch block.
- Display a suitable message confirming that the exception was handled.
- Stop the program.

4. UML Class Diagram

IllegalArgumentException
main(String[] args)
try { exception-causing statement }
catch (ExceptionType e)
print handled message

5. Source Code

```
package observation.exp03;

public class IllegalArgumentException {
    public static void main(String[] args) {

        try {
            Integer.parseInt("abc");
        }
        catch (IllegalArgumentException iae) {
            System.out.println("Invalid argument.");
            System.out.println("IllegalArgumentException is handled successfully.");
        }
    }
}
```

6. Compilation & Execution Commands

```
javac -d . IllegalArgumentException.java
java observation.exp03.IllegalArgumentException
```

7. Output

```
Invalid argument.
IllegalArgumentException is handled successfully.
```

8. Result

The program successfully demonstrates handling of IllegalArgumentException using exception handling in Java.

Experiment 03 — Program 5/10

IllegalStateException

1. Objective / Aim

Use a Scanner after it has been closed and handle IllegalStateException.

2. System Requirements

Hardware: Standard computer/laptop. Software: Java Development Kit (JDK 8 or later), command prompt/terminal, and a Java-compatible editor or IDE.

3. Algorithm / Logic Description

- Start the program.
- Place the statement that may cause the exception inside a try block.
- Execute the statement that produces IllegalStateException.
- Catch IllegalStateException using the corresponding catch block.
- Display a suitable message confirming that the exception was handled.
- Stop the program.

4. UML Class Diagram

IllegalStateException
main(String[] args)
try { exception-causing statement }
catch (ExceptionType e)
print handled message

5. Source Code

```
package observation.exp03;
import java.util.*;

public class IllegalStateException {
    public static void main(String[] args) {

        try {
            Scanner sc = new Scanner(System.in);

            sc.close();

            sc.nextInt();
        }
        catch (IllegalStateException ise) {
            System.out.println("Scanner is already closed.");
            System.out.println("IllegalStateException is handled successfully.");
        }
    }
}
```

6. Compilation & Execution Commands

```
javac -d . IllegalStateException.java
java observation.exp03.IllegalStateException
```

7. Output

```
Scanner is already closed.
IllegalStateException is handled successfully.
```

8. Result

The program successfully demonstrates handling of `IllegalStateException` using exception handling in Java.

Experiment 03 — Program 6/10

IndexOutOfBoundsException

1. Objective / Aim

Access an invalid ArrayList index and handle IndexOutOfBoundsException.

2. System Requirements

Hardware: Standard computer/laptop. Software: Java Development Kit (JDK 8 or later), command prompt/terminal, and a Java-compatible editor or IDE.

3. Algorithm / Logic Description

- Start the program.
- Place the statement that may cause the exception inside a try block.
- Execute the statement that produces IndexOutOfBoundsException.
- Catch IndexOutOfBoundsException using the corresponding catch block.
- Display a suitable message confirming that the exception was handled.
- Stop the program.

4. UML Class Diagram

IndexOutOfBoundsException
main(String[] args)
try { exception-causing statement }
catch (ExceptionType e)
print handled message

5. Source Code

```
package observation.exp03;
import java.util.*;

public class IndexOutOfBoundsException {
    public static void main(String[] args) {

        try {
            ArrayList<Integer> list = new ArrayList<>();

            list.add(10);
            list.add(20);

            System.out.println(list.get(5));
        }
        catch (IndexOutOfBoundsException ioobe) {
            System.out.println("Invalid index.");
            System.out.println("IndexOutOfBoundsException is handled successfully.");
        }
    }
}
```

6. Compilation & Execution Commands

```
javac -d . IndexOutOfBoundsException.java
java observation.exp03.IndexOutOfBoundsException
```

7. Output

```
Invalid index.
IndexOutOfBoundsException is handled successfully.
```

8. Result

The program successfully demonstrates handling of `IndexOutOfBoundsException` using exception handling in Java.

Experiment 03 — Program 7/10

NegativeArraySizeException

1. Objective / Aim

Create an array with a negative size and handle NegativeArraySizeException.

2. System Requirements

Hardware: Standard computer/laptop. Software: Java Development Kit (JDK 8 or later), command prompt/terminal, and a Java-compatible editor or IDE.

3. Algorithm / Logic Description

- Start the program.
- Place the statement that may cause the exception inside a try block.
- Execute the statement that produces NegativeArraySizeException.
- Catch NegativeArraySizeException using the corresponding catch block.
- Display a suitable message confirming that the exception was handled.
- Stop the program.

4. UML Class Diagram

NegativeArraySizeException
main(String[] args)
try { exception-causing statement }
catch (ExceptionType e)
print handled message

5. Source Code

```
package observation.exp03;

public class NegativeArraySizeException {
    public static void main(String[] args) {

        try {
            int ar[] = new int[-5];
        }
        catch (NegativeArraySizeException nase) {
            System.out.println("Array size cannot be negative.");
            System.out.println("NegativeArraySizeException is handled successfully.");
        }
    }
}
```

6. Compilation & Execution Commands

```
javac -d . NegativeArraySizeException.java
java observation.exp03.NegativeArraySizeException
```

7. Output

```
Array size cannot be negative.
NegativeArraySizeException is handled successfully.
```

8. Result

The program successfully demonstrates handling of NegativeArraySizeException using exception handling in Java.

Experiment 03 — Program 8/10

NullPointerException

1. Objective / Aim

Call a method using a null String reference and handle NullPointerException.

2. System Requirements

Hardware: Standard computer/laptop. Software: Java Development Kit (JDK 8 or later), command prompt/terminal, and a Java-compatible editor or IDE.

3. Algorithm / Logic Description

- Start the program.
- Place the statement that may cause the exception inside a try block.
- Execute the statement that produces NullPointerException.
- Catch NullPointerException using the corresponding catch block.
- Display a suitable message confirming that the exception was handled.
- Stop the program.

4. UML Class Diagram

NullPointerException
main(String[] args)
try { exception-causing statement }
catch (ExceptionType e)
print handled message

5. Source Code

```
package observation.exp03;

public class NullPointerException {
    public static void main(String[] args) {

        try {
            String name = null;
            int x = name.length();
        }
        catch (NullPointerException npe) {
            System.out.println("Cannot find the length of a null String.");
            System.out.println("NullPointerException is handled successfully.");
        }
    }
}
```

6. Compilation & Execution Commands

```
javac -d . NullPointerException.java
java observation.exp03.NullPointerException
```

7. Output

```
Cannot find the length of a null String.
NullPointerException is handled successfully.
```

8. Result

The program successfully demonstrates handling of NullPointerException using exception handling in Java.

Experiment 03 — Program 9/10

NumberFormatException

1. Objective / Aim

Convert a non-numeric String to an integer and handle NumberFormatException.

2. System Requirements

Hardware: Standard computer/laptop. Software: Java Development Kit (JDK 8 or later), command prompt/terminal, and a Java-compatible editor or IDE.

3. Algorithm / Logic Description

- Start the program.
- Place the statement that may cause the exception inside a try block.
- Execute the statement that produces NumberFormatException.
- Catch NumberFormatException using the corresponding catch block.
- Display a suitable message confirming that the exception was handled.
- Stop the program.

4. UML Class Diagram

NumberFormatException
main(String[] args)
try { exception-causing statement }
catch (ExceptionType e)
print handled message

5. Source Code

```
package observation.exp03;

public class NumberFormatException {
    public static void main(String[] args) {

        try {
            String name = "Java";

            int x = Integer.parseInt(name);
        }
        catch (NumberFormatException nfe) {
            System.out.println("Cannot convert an invalid String into a number.");
            System.out.println("NumberFormatException is handled successfully.");
        }
    }
}
```

6. Compilation & Execution Commands

```
javac -d . NumberFormatException.java
java observation.exp03.NumberFormatException
```

7. Output

```
Cannot convert an invalid String into a number.
NumberFormatException is handled successfully.
```

8. Result

The program successfully demonstrates handling of `NumberFormatException` using exception handling in Java.

Experiment 03 — Program 10/10

StringIndexOutOfBoundsException

1. Objective / Aim

Access an invalid String position and handle StringIndexOutOfBoundsException.

2. System Requirements

Hardware: Standard computer/laptop. Software: Java Development Kit (JDK 8 or later), command prompt/terminal, and a Java-compatible editor or IDE.

3. Algorithm / Logic Description

- Start the program.
- Place the statement that may cause the exception inside a try block.
- Execute the statement that produces StringIndexOutOfBoundsException.
- Catch StringIndexOutOfBoundsException using the corresponding catch block.
- Display a suitable message confirming that the exception was handled.
- Stop the program.

4. UML Class Diagram

StringIndexOutOfBoundsException
main(String[] args)
try { exception-causing statement }
catch (ExceptionType e)
print handled message

5. Source Code

```
package observation.exp03;

public class StringIndexOutOfBoundsException {
    public static void main(String[] args) {

        try {
            String name = "Java";

            int x = name.charAt(6);
        }
        catch (StringIndexOutOfBoundsException siobe) {
            System.out.println("Cannot access an invalid position in a String.");
            System.out.println("StringIndexOutOfBoundsException is handled successfully.");
        }
    }
}
```

6. Compilation & Execution Commands

```
javac -d . StringIndexOutOfBoundsException.java
java observation.exp03.StringIndexOutOfBoundsException
```

7. Output

```
Cannot access an invalid position in a String.
StringIndexOutOfBoundsException is handled successfully.
```

8. Result

The program successfully demonstrates handling of `StringIndexOutOfBoundsException` using exception handling in Java.